RAG-Based PayanarssType Retrieval System

Overview

You have pre-created 1,300+ PayanarssTypes covering all business functionalities. The RAG system retrieves the **correct existing PayanarssTypes** based on natural language business requirements, without generating new ones.

Architecture Flow

```
┌─────────────────────────────────────────────────────────────┐
│              USER BUSINESS REQUIREMENT PROMPT               │
│  "I need client management for my restaurant"               │
└─────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌─────────────────────────────────────────────────────────────┐
│          EMBEDDING GENERATION (Claude/OpenAI)               │
│  Convert prompt to vector embedding                         │
└─────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌─────────────────────────────────────────────────────────────┐
│       VECTOR DATABASE SEARCH (ChromaDB/Pinecone)            │
│  Search 1,300+ pre-stored PayanarssTypes                    │
│  Find semantically similar PayanarssTypes                   │
└─────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌─────────────────────────────────────────────────────────────┐
│           CLAUDE AI RANKING & FILTERING                     │
│  - Analyze retrieved PayanarssTypes                         │
│  - Rank by relevance to business requirement                │
│  - Filter out irrelevant ones                               │
│  - Explain why each is relevant                             │
└─────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌─────────────────────────────────────────────────────────────┐
│         RETURN MATCHED PAYANARSSTYPES                       │
│  Present user with relevant pre-built modules               │
│  User selects which ones to activate                        │
└─────────────────────────────────────────────────────────────┘
                              │
                              ↓
┌─────────────────────────────────────────────────────────────┐
│           ACTIVATE SELECTED MODULES                         │
│  - Deploy selected PayanarssTypes to user's instance        │
```

```
│  - Create database tables                    │
│  - Generate UI screens                       │
│  - Configure workflows                        │
└──────────────────────────────────────────────┘
```

Step 1: One-Time Setup - Store All PayanarssTypes in Vector DB

1.1 Load Your 1,300+ PayanarssTypes

```python
```

```python
import json
import chromadb
from chromadb.config import Settings

class PayanarssTypeVectorStore:
    def __init__(self):
        self.client = chromadb.Client(Settings(
            chroma_db_impl="duckdb+parquet",
            persist_directory="./maa_erp_payanarss_db"
        ))

        # Create collection for PayanarssTypes
        self.collection = self.client.get_or_create_collection(
            name="payanarss_types_repository",
            metadata={"description": "1,300+ pre-built PayanarssTypes for all business scenarios"}
        )

    def load_all_payanarsstypes(self, json_file_path):
        """
        One-time load of all your PayanarssTypes into vector database
        """
        # Load your VanakkamPayanarssTypesV1.json
        with open(json_file_path, 'r') as f:
            all_types = json.load(f)

        documents = []
        metadatas = []
        ids = []

        for pt in all_types:
            # Create rich searchable text for embedding
            searchable_text = self.create_searchable_text(pt)

            # Metadata for filtering
            metadata = {
                "id": pt['Id'],
                "name": pt['Name'],
                "parent_id": pt['ParentId'],
                "type_id": pt['PayanarssTypeId'],
                "description": pt.get('Description', ''),
                "module": self.extract_module_name(pt),
                "industry": self.extract_industry(pt),
                "complexity": self.calculate_complexity(pt)
            }

            documents.append(searchable_text)
```

```
        metadatas.append(metadata)
        ids.append(pt['Id'])

    # Store in ChromaDB with embeddings
    self.collection.add(
        documents=documents,
        metadatas=metadatas,
        ids=ids
    )

    print(f"✅ Loaded {len(all_types)} PayanarssTypes into vector database")

def create_searchable_text(self, payanarss_type):
    """
    Create rich text combining all searchable fields
    This text will be embedded for semantic search
    """
    text_parts = [
        f"Name: {payanarss_type['Name']}",
        f"Description: {payanarss_type.get('Description', '')}",
    ]

    # Add parent information for context
    if payanarss_type['ParentId'] != payanarss_type['Id']:
        text_parts.append(f"Part of module: {payanarss_type['ParentId']}")

    # Add type information
    type_names = {
        "10000000000000000000000000000001": "Table/Entity",
        "10000000000000000000000000000002": "Child Table/Relation",
        "10000000000000000000000000000003": "Lookup/Dropdown",
        "10000000000000000000000000000004": "Module/Group",
    }
    type_name = type_names.get(payanarss_type['PayanarssTypeId'], 'Unknown')
    text_parts.append(f"Type: {type_name}")

    return " | ".join(text_parts)

def extract_module_name(self, pt):
    """Extract module name from hierarchy"""
    # Logic to determine module (e.g., from Id prefix or ParentId)
    if 'PM' in pt['Id']:
        return "Property Management"
    elif 'PP' in pt['Id']:
        return "Project Planning"
    elif 'CM' in pt['Id']:
        return "Client Management"
```

```python
    # Add more patterns
    return "Core"

def extract_industry(self, pt):
    """Determine which industry this PayanarssType serves"""
    name_lower = pt['Name'].lower()
    description_lower = pt.get('Description', '').lower()

    industries = []

    # Check for industry keywords
    if any(word in name_lower or word in description_lower
           for word in ['hotel', 'booking', 'reservation', 'guest', 'room']):
        industries.append("Hospitality")

    if any(word in name_lower or word in description_lower
           for word in ['property', 'tenant', 'lease', 'rental']):
        industries.append("Real Estate")

    if any(word in name_lower or word in description_lower
           for word in ['project', 'task', 'milestone', 'resource']):
        industries.append("Project Management")

    if any(word in name_lower or word in description_lower
           for word in ['design', 'interior', 'quotation', 'client']):
        industries.append("Interior Design")

    if any(word in name_lower or word in description_lower
           for word in ['restaurant', 'kitchen', 'meal', 'menu', 'recipe']):
        industries.append("Restaurant")

    return ",".join(industries) if industries else "General"

def calculate_complexity(self, pt):
    """Estimate complexity for better matching"""
    complexity = 1

    # More attributes = more complex
    if isinstance(pt.get('Attributes'), list):
        complexity += len(pt['Attributes']) * 0.5

    # Longer description = more complex
    if pt.get('Description'):
        complexity += len(pt['Description']) / 100

    return min(complexity, 5)  # Cap at 5
```

1.2 Initialize the Vector Database

```python
# One-time setup script
def initialize_payanarsstype_repository():
    store = PayanarssTypeVectorStore()

    # Load your master PayanarssTypes file
    store.load_all_payanarsstypes('./VanakkamPayanarssTypesV1.json')

    # Load Interior Design specific types
    store.load_all_payanarsstypes('./InteriorDesignPayanarssTypes.json')

    # Load Building Management types
    store.load_all_payanarsstypes('./BuildingManagementTypes.json')

    # Load Project Planning types
    store.load_all_payanarsstypes('./ProjectPlanningTypes.json')

    print("✅ PayanarssType repository initialized and ready for retrieval!")

# Run once
initialize_payanarsstype_repository()
```

Step 2: Retrieval System - Find Relevant PayanarssTypes

```python
```

```python
class PayanarssTypeRetriever:
    def __init__(self):
        self.vector_store = PayanarssTypeVectorStore()
        self.collection = self.vector_store.collection

    def retrieve_by_prompt(self, user_prompt, top_k=20, filters=None):
        """
        Main retrieval method - finds relevant PayanarssTypes based on prompt

        Args:
            user_prompt: Natural language business requirement
            top_k: Number of results to return
            filters: Optional metadata filters (industry, module, etc.)

        Returns:
            List of matching PayanarssTypes with relevance scores
        """
        # Query vector database
        results = self.collection.query(
            query_texts=[user_prompt],
            n_results=top_k,
            where=filters  # e.g., {"industry": "Restaurant"}
        )

        # Structure the results
        matched_types = []
        for i, doc_id in enumerate(results['ids'][0]):
            matched_types.append({
                'id': doc_id,
                'metadata': results['metadatas'][0][i],
                'document': results['documents'][0][i],
                'distance': results['distances'][0][i] if 'distances' in results else None,
                'rank': i + 1
            })

        return matched_types

    def retrieve_with_claude_ranking(self, user_prompt, industry=None, top_k=20):
        """
        Enhanced retrieval with Claude AI for intelligent ranking
        """
        # Step 1: Vector search
        filters = {"industry": {"$contains": industry}} if industry else None
        initial_results = self.retrieve_by_prompt(user_prompt, top_k=top_k, filters=filters)

        # Step 2: Use Claude to rank and explain relevance
```

```
        ranked_results = self.claude_rank_results(user_prompt, initial_results)

        return ranked_results

    def claude_rank_results(self, user_prompt, results):
        """
        Use Claude to intelligently rank and explain matches
        """
        from anthropic import Anthropic

        client = Anthropic(api_key="your-api-key")

        # Prepare context for Claude
        results_summary = "\n".join([
            f"{i+1}. {r['metadata']['name']} - {r['metadata']['description']}"
            for i, r in enumerate(results[:10])  # Send top 10 to Claude
        ])

        prompt = f"""
User Business Requirement:
"{user_prompt}"

Retrieved PayanarssTypes (pre-built modules):
{results_summary}

Your task:
1. Rank these PayanarssTypes by relevance to the user's requirement (1 = most relevant)
2. For each relevant item, explain WHY it matches the requirement
3. Identify if any critical modules are missing from the results
4. Suggest combinations of modules that work well together

Output as JSON:
{{
  "ranked_modules": [
    {{
      "rank": 1,
      "module_name": "...",
      "relevance_score": 0.95,
      "explanation": "...",
      "use_case": "..."
    }}
  ],
  "missing_modules": ["..."],
  "recommended_combinations": [["module1", "module2", "module3"]]
}}
"""
```

```
    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=2000,
        messages=[{"role": "user", "content": prompt}]
    )

    # Parse Claude's response
    import json
    claude_analysis = json.loads(response.content[0].text)

    return claude_analysis
```

Step 3: User-Facing Retrieval API

```
php
```

```php
<?php

namespace App\Http\Controllers\API;

use App\Services\PayanarssTypeRetriever;
use Illuminate\Http\Request;

class PayanarssTypeDiscoveryController extends Controller
{
    private $retriever;

    public function __construct()
    {
        $this->retriever = new PayanarssTypeRetriever();
    }

    /**
     * Main endpoint: User describes what they need, get matching modules
     *
     * POST /api/v1/discover-modules
     * Body: {
     *   "prompt": "I need client management for my restaurant",
     *   "industry": "Restaurant",
     *   "preferred_modules": ["Client Management", "Billing"]
     * }
     */
    public function discoverModules(Request $request)
    {
        $userPrompt = $request->input('prompt');
        $industry = $request->input('industry');

        // Call Python retrieval service via HTTP or direct Python execution
        $pythonScript = base_path('python/retrieve_payanarss.py');

        $command = sprintf(
            'python3 %s --prompt=%s --industry=%s --top_k=20',
            escapeshellarg($pythonScript),
            escapeshellarg($userPrompt),
            escapeshellarg($industry)
        );

        $output = shell_exec($command);
        $results = json_decode($output, true);

        return response()->json([
            'success' => true,
```

```
        'user_prompt' => $userPrompt,
        'matched_modules' => $results['ranked_modules'],
        'total_found' => count($results['ranked_modules']),
        'recommendations' => $results['recommended_combinations'],
        'missing_modules' => $results['missing_modules']
    ]);
}

/**
 * Get full PayanarssType details for selected modules
 *
 * POST /api/v1/get-payanarss-details
 * Body: {
 *   "module_ids": ["PP1000001-...", "PP2000001-..."]
 * }
 */
public function getPayanarssDetails(Request $request)
{
    $moduleIds = $request->input('module_ids');

    // Load full PayanarssType JSON from database or file
    $fullDetails = [];
    foreach ($moduleIds as $id) {
        $payanarssType = $this->loadPayanarssTypeById($id);

        // Get all related child tables
        $relatedTypes = $this->loadRelatedPayanarssTypes($id);

        $fullDetails[] = [
            'main' => $payanarssType,
            'related' => $relatedTypes,
            'estimated_tables': count($relatedTypes),
            'estimated_screens': $this->calculateScreens($payanarssType, $relatedTypes)
        ];
    }

    return response()->json([
        'success' => true,
        'modules' => $fullDetails
    ]);
}

/**
 * Activate selected modules for a user
 *
 * POST /api/v1/activate-modules
 * Body: {
```

```
     *   "user_id": "user-123",
     *   "module_ids": ["PP1000001-...", "PP2000001-..."],
     *   "configuration": {...}
     * }
     */
    public function activateModules(Request $request)
    {
        $userId = $request->input('user_id');
        $moduleIds = $request->input('module_ids');

        // Deploy selected PayanarssTypes to user's instance
        foreach ($moduleIds as $moduleId) {
            // 1. Create database tables
            $this->createDatabaseTables($moduleId, $userId);

            // 2. Generate UI components
            $this->generateUIComponents($moduleId, $userId);

            // 3. Set up workflows
            $this->configureWorkflows($moduleId, $userId);

            // 4. Apply permissions
            $this->setPermissions($moduleId, $userId);
        }

        return response()->json([
            'success' => true,
            'message' => 'Modules activated successfully',
            'modules_activated' => count($moduleIds),
            'dashboard_url' => "/dashboard/{$userId}"
        ]);
    }
}
```

Step 4: Python Retrieval Script

```python
```

```python
#!/usr/bin/env python3
# python/retrieve_payanarss.py

import argparse
import json
import sys
from anthropic import Anthropic

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--prompt', required=True)
    parser.add_argument('--industry', default=None)
    parser.add_argument('--top_k', type=int, default=20)

    args = parser.parse_args()

    # Initialize retriever
    retriever = PayanarssTypeRetriever()

    # Retrieve and rank
    results = retriever.retrieve_with_claude_ranking(
        user_prompt=args.prompt,
        industry=args.industry,
        top_k=args.top_k
    )

    # Output JSON for PHP to consume
    print(json.dumps(results, indent=2))

if __name__ == '__main__':
    main()
```

Example Usage Scenarios

Scenario 1: Restaurant Owner

```javascript
```

```
// User Input
{
  "prompt": "I need to manage my VIP private kitchen - clients, meals, bookings, billing",
  "industry": "Restaurant"
}

// RAG System Response
{
  "matched_modules": [
    {
      "rank": 1,
      "name": "Client Management",
      "relevance_score": 0.98,
      "explanation": "Essential for tracking VIP clients with contact info, preferences, and history",
      "payanarss_ids": ["CM1000001-...", "CM1000002-...", "CM1000003-..."],
      "tables_included": ["Client_Master", "Client_Contacts", "Client_Classification"],
      "estimated_setup_time": "5 minutes"
    },
    {
      "rank": 2,
      "name": "Meal Management",
      "relevance_score": 0.96,
      "explanation": "Track meal plans, recipes, ingredients for your VIP kitchen",
      "payanarss_ids": ["ML1000001-...", "ML1000002-..."],
      "tables_included": ["Meal_Master", "Meal_Recipes", "Ingredients"],
      "estimated_setup_time": "3 minutes"
    },
    {
      "rank": 3,
      "name": "Booking Management",
      "relevance_score": 0.95,
      "explanation": "Schedule and manage VIP dining bookings with conflict detection",
      "payanarss_ids": ["BK1000001-...", "BK1000002-..."],
      "tables_included": ["Bookings", "Booking_Calendar", "Booking_Status"],
      "estimated_setup_time": "4 minutes"
    },
    {
      "rank": 4,
      "name": "Billing & Invoicing",
      "relevance_score": 0.92,
      "explanation": "Generate invoices, track payments, manage credit terms",
      "payanarss_ids": ["BL1000001-...", "BL1000002-..."],
      "tables_included": ["Invoices", "Payments", "Credit_Terms"],
      "estimated_setup_time": "3 minutes"
    }
  ],
```

```
  "recommended_combinations": [
    {
      "name": "Complete VIP Kitchen Solution",
      "modules": ["Client Management", "Meal Management", "Booking Management", "Billing & Invoicing"],
      "total_tables": 15,
      "total_setup_time": "15 minutes"
    }
  ],
  "missing_modules": [],
  "total_results": 4
}
```


Scenario 2: Interior Designer

```javascript
```

```
// User Input
{
  "prompt": "I do residential interior design - need project tracking, quotations, vendor management",
  "industry": "Interior Design"
}

// RAG System Response
{
  "matched_modules": [
    {
      "rank": 1,
      "name": "Project Management",
      "relevance_score": 0.97,
      "explanation": "Complete project lifecycle from inquiry to completion",
      "payanarss_ids": ["PP1000001-...", ...],
      "features": [
        "Project timeline & milestones",
        "Task assignment",
        "Budget vs Actual tracking",
        "Document management"
      ]
    },
    {
      "rank": 2,
      "name": "Quotation & Estimation",
      "relevance_score": 0.95,
      "explanation": "Generate detailed quotations with materials, labor, and service costs",
      "payanarss_ids": ["QT1000001-...", ...],
      "features": [
        "Itemized quotes",
        "Discount management",
        "Quote-to-project conversion",
        "Approval workflow"
      ]
    },
    {
      "rank": 3,
      "name": "Supplier & Vendor Management",
      "relevance_score": 0.93,
      "explanation": "Track suppliers, compare prices, manage procurement",
      "payanarss_ids": ["SV1000001-...", ...],
      "features": [
        "Supplier database",
        "Performance metrics",
        "Purchase orders",
        "Delivery tracking"
```

```
      ]
    }
  ],
  "recommended_combinations": [
    {
      "name": "Interior Design Complete Suite",
      "modules": [
        "Client Management",
        "Project Management",
        "Design & Consultation",
        "Quotation & Estimation",
        "Material Management",
        "Supplier Management",
        "Execution & Installation",
        "Billing & Payments"
      ],
      "matches_document": "Interior Design ERP - Business Modules.pdf",
      "total_tables": 45,
      "total_setup_time": "25 minutes"
    }
  ]
}
```


Scenario 3: Property Manager

```javascript
```

```
// User Input
{
  "prompt": "Manage rental properties - tenants, leases, rent collection, maintenance",
  "industry": "Real Estate"
}

// RAG System Response
{
  "matched_modules": [
    {
      "rank": 1,
      "name": "Property Management",
      "relevance_score": 0.99,
      "explanation": "Core property information, units, floors, amenities tracking",
      "payanarss_ids": ["PM1000001-...", "PM2000001-...", ...],
      "tables": 18
    },
    {
      "rank": 2,
      "name": "Tenant Management",
      "relevance_score": 0.98,
      "explanation": "Track tenants, contact info, screening, history",
      "payanarss_ids": ["TM1000001-...", ...],
      "tables": 6
    },
    {
      "rank": 3,
      "name": "Lease Management",
      "relevance_score": 0.97,
      "explanation": "Lease agreements, renewal tracking, terms management",
      "payanarss_ids": ["LM1000001-...", ...],
      "tables": 8
    },
    {
      "rank": 4,
      "name": "Rent Collection & Billing",
      "relevance_score": 0.96,
      "explanation": "Automated rent invoicing, payment tracking, late fees",
      "payanarss_ids": ["RC1000001-...", ...],
      "tables": 7
    }
  ],
  "recommended_combinations": [
    {
      "name": "Complete Rental Management",
      "modules": [
```

```
      "Property Management",
      "Tenant Management",
      "Lease Management",
      "Rent Collection & Billing",
      "Maintenance Management",
      "Document Management"
    ],
    "total_tables": 52,
    "total_setup_time": "30 minutes"
  }
 ]
}
```

Benefits of RAG Approach

✅ **No Code Generation** - Use your pre-built, tested PayanarssTypes ✅ **Instant Discovery** - Find relevant modules in seconds ✅ **Semantic Understanding** - Matches business intent, not just keywords ✅ **Intelligent Recommendations** - Claude suggests module combinations ✅ **Version Control** - All PayanarssTypes are versioned in your repository ✅ **Quality Assured** - You've tested everything beforehand ✅ **Fast Deployment** - Select and activate in minutes

---

Data Flow Summary

```
1. YOU create PayanarssTypes manually → Store in JSON files
2. ONE-TIME: Load all PayanarssTypes into Vector DB (ChromaDB)
3. USER describes need in natural language
4. SYSTEM searches vector DB for semantically similar PayanarssTypes
5. CLAUDE ranks results and explains relevance
6. USER selects which modules to activate
7. SYSTEM deploys selected PayanarssTypes to user's instance
```

This is a **pure retrieval system** - your PayanarssTypes stay intact, and users discover them through intelligent search! 🎯